Systems Engineering Agentic Cookbook

Three ingredients. One repeatable recipe for reliable healthcare systems.

SYSTEMS-FIRST	SPEC-DRIVEN	AI-ASSISTED
Lifecycle Thinking	Development	Execution

Justin | Lab651 & Recursive Awesome | INCOSE Healthcare Systems Conference

Building Software for Regulated Industries

ABOUT THE SPEAKER

25 Years. Regulated Systems. Real Codebases.

LAB651

Product-minded & AI assisted engineers · Mobile · Cloud

RECURSIVE AWESOME

Medical · Insurance · Industrial – AI Strategy

applied ai

Non-Profit: Building the next generation of AI Leaders

25 yrs

Building production software, many in regulated industries

IoT + AI

Embedded systems to cloud to LLM pipelines

Educator

Graduate AI/ML & IoT — University of St. Thomas

Thought Leader

Founder + CEO & Thought Leader on Artificial Intelligence

Today's talk is built on production experience — not theory.

2:47 AM. Room 412. Insulin overdue.

The alert system shows no overdue alerts. The code looks clean. Tests pass.

But a developer, six weeks ago, copy-pasted a date comparison. Timezone handling is wrong for daylight saving time. There is no requirement that specified UTC. No test covering timezone edge cases. No audit log to surface the discrepancy.

Just a patient. And silence.

The root cause wasn't the developer. It wasn't the timezone. It was the absence of structure.

No requirement captured the constraint. No test verified it. No trace connected hazard to code.

THE PROBLEM

AI Amplifies What's Already There

"VIBE CODING" — Fast, Fragile, Untraceable

- ✗ No explicit requirements → intent lives only in developer's head
- ✗ AI generates confident-looking code with no verified behavior
- ✗ Tests validate the implementation, not the requirement
- ✗ Zero traceability: requirement → design → code → test
- ✗ Audit trail: nonexistent

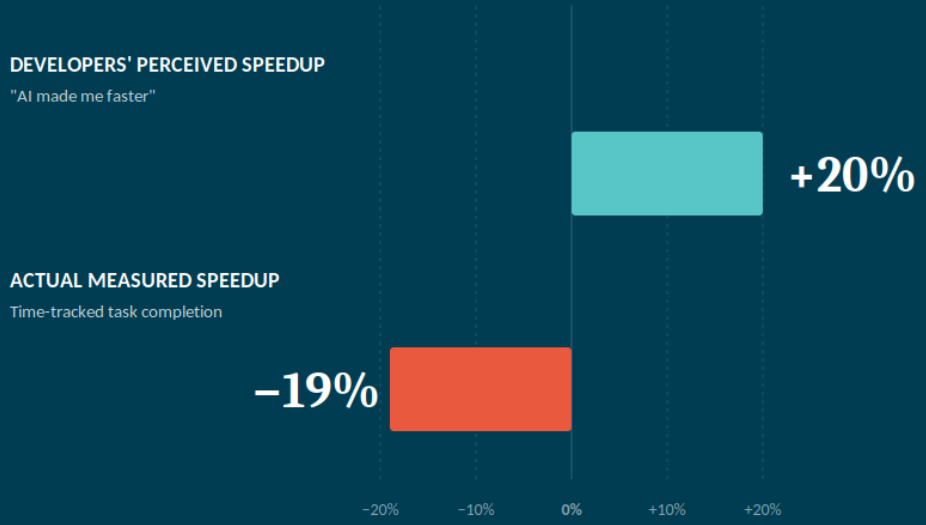
> *Imagine a junior engineer and you just say: 'refactor this.' That's a huge challenge — they need context, guardrails, rules. That's exactly how I think about working with AI.*

In regulated industries, speed without structure is not a feature. It is a liability.

THE PRODUCTIVITY PARADOX

Developers Think AI Makes Them Faster

METR RANDOMIZED CONTROLLED TRIAL · 246 REAL-WORLD DEV TASKS



A 39-point reality gap.

Experienced developers working on repos they knew well took **19% longer with AI tools —** while reporting they had been sped up.

Even after seeing their own slowdown in the data, the same developers still believed AI had made them faster.

Self-report ≠ measurement.

Source: METR, "Measuring the Impact of AI on Experienced Open-Source Developer Productivity," 16 senior devs · 246 tasks · randomized AI-allowed vs. AI-disallowed (2025, follow-up Feb 2026)

THE PRODUCTIVITY PARADOX

Speed Goes Up. Quality Quietly Collapses.

GITCLEAR · 211 MILLION LINES OF CODE · 2020 – 2024

REFACTORING (HEALTHY CHANGES)

–60%

25% → under 10%

Refactoring's share of code changes more than halved between 2021 and 2024.

Engineers stopped reshaping existing code. They added new lines on top of it.

DUPLICATED CODE BLOCKS

8×

Copy-pasted code: 8.3% → 12.3%

Eightfold rise in duplicated blocks — the first year copy-paste outpaced reuse.

DRY violations multiply maintenance cost and propagate the same bug to every clone.

CODE CHURN (REWRITTEN < 2 WEEKS)

2×

3.1% (2020) → 5.7%+ (2024)

Code rewritten or reverted within two weeks nearly doubled — a leading defect signal.

More "almost right" code. More rework. Productivity gain absorbed by repair cost.

Source: GitClear, "AI Copilot Code Quality: 2025 Research" — 211M changed lines, thousands of repos (Jan 2020 – Dec 2024). Echoed by Google's DORA 2024 report and CodeRabbit's 2025 PR analysis.

SECURITY IS A CONCERN

Vulnerabilities Double in One Year

APIIRO · FORTUNE 50 ENTERPRISE STUDY · DEC 2024 – JUN 2025

10×

surge in security findings

Monthly findings climbed from a baseline in December 2024 to over 10,000 per month by June 2025 — a 10× increase in just six months.

WHERE THE RISK IS COMPOUNDING

+322% — **Privilege-escalation paths**
Auth checks without authorization checks.

+153% — **Architectural design flaws**
10–100× more expensive to remediate than bugs.

2.74× — **More vulnerabilities per 1,000 LOC**
Veracode tested 100+ LLMs · 45% security failure rate.

+40% — **Repos with exposed secrets**
Hardcoded credentials, API keys, tokens.

Sources: Apiiro, "4× Velocity, 10× Vulnerabilities" — Deep Code Analysis across tens of thousands of Fortune 50 repos (2025) · Veracode, "GenAI Code Security Report" (2025).

REGULATORY REALITY

The Standards Don't Care How Fast You Shipped

IEC 62304

Software Lifecycle
for Medical Devices

- Software safety classification
- Requirements → architecture → detailed design
- Risk-driven verification

ISO 14971

Risk Management
for Medical Devices

- Hazard identification → risk estimation
- Risk control measures documented
- Residual risk acceptability

21 CFR Part 11

Electronic Records
& Signatures (FDA)

- Audit trails required & immutable
- Access controls documented
- System validation documented

All three standards require: documented requirements · traceable design · verifiable behavior · immutable audit records

THE FRAMEWORK

Three Practices. One Disciplined Engineering Model.

01

SYSTEMS-FIRST ENGINEERING

Connect every decision to the full system lifecycle — from requirement to design to verification to post-deployment evolution.

- Risk-driven requirements
- Architectural integrity over time

02

SPEC-DRIVEN DEVELOPMENT

Structure before syntax. The specification is the primary engineering artifact. Code is its deterministic expression.

- REQ-NNN format requirements
- State machine design before code

03

AI-ASSISTED EXECUTION

AI operates within defined constraints — not instead of them. The spec is the AI's instruction manual and its safety boundary.

- Requirement-aware code generation
- Traced test generation by REQ

Speed with rigor. Compliance by design. Architectural integrity that survives the full system lifecycle.

DEMO SYSTEM

Medication Dose Alert Monitor (MDAM)

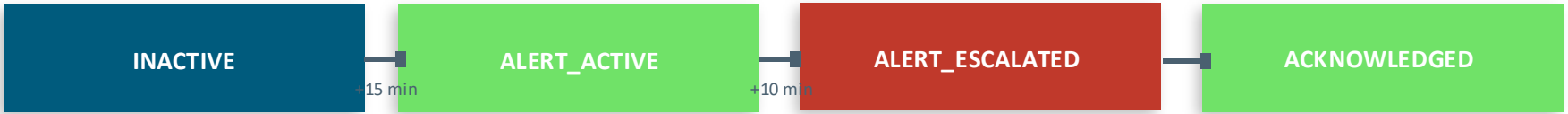
What It Does

Monitors scheduled medication administration. Detects overdue doses. Generates care alerts with escalation. Requires human acknowledgment. Maintains an immutable audit log of every state change.

Why This System

- ✓ Timing constraints
- ✓ State transitions
- ✓ Safety-critical behavior
- ✓ Human-in-the-loop
- ✓ Compliance-relevant audit

THE ONE FEATURE: Overdue Dose Alert with Escalation



Every state transition is logged with: prior state · new state · triggering event · UTC timestamp · actor ID

This audit log is immutable. It IS your verification evidence under IEC 62304 and 21 CFR Part 11.

STEP 1: REQUIREMENTS

Spec-First: Making Intent Explicit

REQ-001	System SHALL detect any scheduled dose with no administration record within a configurable threshold (default 15 min) and emit an ALERT_ACTIVE event.
REQ-003	System SHALL NOT emit more than one ALERT_ACTIVE event per unique dose event, regardless of evaluation frequency.
REQ-004	System SHALL emit an ALERT_ESCALATED event if an ALERT_ACTIVE event remains unacknowledged for a configurable escalation period (default 10 min).
REQ-006	System SHALL record all state transitions with: prior state, new state, event, UTC timestamp, actor ID. Audit log SHALL be immutable.
REQ-009	On any internal error during detection or escalation, the system SHALL surface a SYSTEM_ALERT to the operator and SHALL NOT suppress or swallow the error silently.
REQ-010	All detection and escalation functions SHALL be deterministic: identical inputs SHALL always produce identical outputs with no side effects.

These requirements are AI-reviewable, AI-implementable, and AI-traceable — because they are explicit and structured.

THE CORE CONTRAST

Same AI. Completely Different Engineering.

❌ VIBE CODING PROMPT

```
"Build me a medication alert system that
notifies nurses when doses are late"
```

What gets generated:

- setTimeout-based timing (no UTC, no injection)
- Hardcoded email function, no role map
- No state machine — flag variable
- No audit log, no actor tracking

Vibe coding has a place: exploration, research, quick bug hunts.

✅ SPEC-DRIVEN PROMPT

```
"Implement DoseStateEngine satisfying REQ-001 through
REQ-012. Pure function. Injected timestamps. JSDoc
tracing each method to requirement ID."
```

What gets generated:

- ✓ Pure function — no hidden state
- ✓ Injected timestamps (deterministic, testable)
- ✓ Explicit state machine — all transitions typed
- ✓ Immutable audit entry on every transition
- ✓ JSDoc: @satisfies REQ-001 per method
- ✓ Tests labeled per requirement, boundary cases covered

Spec-driven is non-negotiable when traceability is required.

AI-ASSISTED WORKFLOW

How AI Fits Into a Disciplined Engineering Process

Requirement Review

AI: AI reviews spec for gaps, ambiguities, missing edge cases

↳ *Catches missing edge cases pre-code*

Implementation

AI: AI generates state engine with @satisfies tags linked to each REQ

↳ *Pure functions, ready for verification*

Test Generation

AI: AI generates one test suite section per requirement, boundary cases included

↳ *Tests ARE your verification matrix*

Traceability Matrix

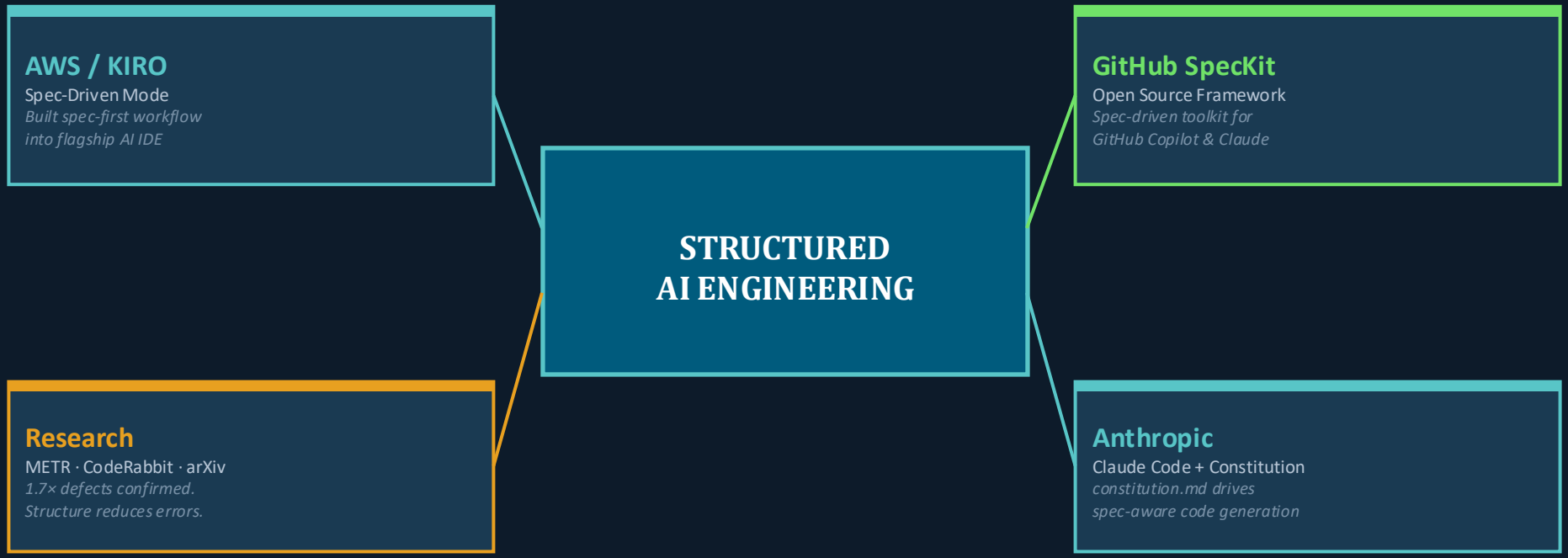
AI: AI generates REQ → function → test → status table from code and tests

↳ *30 seconds, not weeks*

AI is fastest and most reliable when given structure. The spec is the structure.

THE ARGUMENT

The Industry Is Independently Arriving at the Same Answer



AWS · GitHub · Anthropic · Independent Research — all converging: structure before syntax.

THE TOOL

GitHub SpecKit: Structure That AI Can Actually Use

What GitHub SpecKit Is

An open source, spec-driven development framework from GitHub, purpose-built for AI-assisted engineering workflows. SpecKit gives AI the structured context it needs to generate code, tests, and traceability artifacts — rather than guessing at intent from a vague prompt.

- **System Context Block** Named system, version, author, safety classification
- **REQ-NNN Requirements** Functional, constraint, and safety requirements in SHALL language
- **State Machine Definition** Explicit states, transitions, triggers, and guards

LIVE DEMO

We'll now leave the slides.

① **Open MDAM project · run: claude**

② **Walk the spec artifacts — show what four commands produced**

③ **Implement T020 → evaluateDetection() engine**

④ **Implement T021 + T022 → 60 tests by REQ**

⑤ **Implement T034 → traceability matrix**

GitHub SpecKit is open source — github.com/github/spec-kit · Works with GitHub Copilot and other AI assistants

After the demo we return here to close with the traceability matrix and lifecycle argument.

BEFORE THE DEMO — THE COMMANDS THAT GENERATED THESE FILES

This Spec Was Built With Four SpecKit Commands

01 /speckit-constitution

Establish non-negotiable engineering principles: IEC 62304, determinism, @satisfies traceability, immutable audit, fail-safe, strict TypeScript.

PROMPT: "Create governing principles for a safety-critical medication alert monitor"

→ .specify/memory/constitution.md

02 /speckit-specify

Describe WHAT and WHY, not the tech stack. 3 user stories, 12 requirements (REQ-001–REQ-012), state machine, success criteria.

PROMPT: "Overdue Dose Alert with Escalation — OBJECTIVE / BEHAVIOR / CONSTRAINTS / VERIFICATION"

→ specs/001-overdue-dose-alert/spec.md

03 /speckit-plan

TypeScript strict mode, pure functions, injected time, annotations, append-only audit. AI makes 7 architecture decisions from spec alone.

PROMPT: "TypeScript strict mode. Node.js 20 LTS. Vitest. PostgreSQL via adapter."

→ plan.md · data-model.md · research.md

04 /speckit-tasks

35 tasks across 6 phases: types → ports → fakes → state engine → tests (by REQ) → traceability matrix. 100% REQ coverage.

PROMPT: "Sequence tasks as: types → state engine → test suite → traceability matrix"

→ specs/001-overdue-dose-alert/tasks.md

These four commands ran before this talk. The files they generated are what you'll see opened on screen during the demo.

Same workflow. Every time. Repeatable by design.

STEP 4: TRACEABILITY

The AI-Generated Verification Evidence

Req ID	Requirement	Implementing Function	Verifying Test	Status
REQ-001	INACTIVE → ALERT_ACTIVE detection	evaluateDetection()	REQ-001: boundary at 15:00.000	✅
REQ-003	no duplicate ALERT_ACTIVE	evaluateDetection()	REQ-003: duplicate suppression	✅
REQ-006	Immutable audit log	WRITE_AUDIT effect (all engine fns)	REQ-002: audit entry shape	⚠️
REQ-008	Valid ack closes alert, stops escalation	evaluateAcknowledgment()	REQ-008: transitions to ACKNOWLEDGED	⏳
REQ-010	Deterministic transitions	evaluateDetection()	REQ-010: same input → same output	⚠️
REQ-009	Fail-safe: surface SYSTEM_ALERT	withFailSafe()	REQ-009: error path → alert state	⏳

Generated in 30 seconds from the spec + implementation + tests.

In a regulated engagement, this becomes your design verification matrix — traceability as a byproduct of good engineering.

SYSTEMS ENGINEERING CONTEXT

SpecKit Maps Directly to the V-Model.

SPECIFICATION & DESIGN		VERIFICATION & VALIDATION
SYSTEM REQUIREMENTS `/speckit-constitution` Non-negotiable principles · Regulatory standards · Safety constraints	▼	**SYSTEM VALIDATION** **Full quality gate pipeline** pnpm check · tsc · coverage ≥90% · traceability CI
SOFTWARE REQUIREMENTS `/speckit-specify` REQ-NNN requirements · Acceptance scenarios · Boundary conditions	▼	**SOFTWARE INTEGRATION TEST** **Traceability matrix** specs/requirements.md · REQ → function → test → status
ARCHITECTURAL DESIGN `/speckit-plan` Constitution check · Traceability map · Data model · ADRs	▼	**SOFTWARE UNIT VERIFICATION** **Test suite by REQ** 60 tests · 7 describe blocks labeled REQ-NNN · injected time
DETAILED DESIGN `/speckit-tasks` 35 tasks · File paths · REQ-NNN per task · Parallel markers	▼	**CODE REVIEW** **@satisfies annotations** Every export annotated · CI traceability check enforced
	IMPLEMENTATION **(Task References)**	

Same artifacts. Same rigor. Now generated in minutes instead of weeks.

THE FULL LIFECYCLE

This Approach Holds. Through Every Phase.

CONCEPT	REQUIREMENTS	DESIGN	IMPLEMENTATION	VERIFICATION	EVOLUTION
Risk ID	REQ-NNN authored	State machines drawn	AI generates within spec	Test run = V&V matrix	Spec updated first
Stakeholder needs	Constraints defined	Architecture decided	Code traced to REQ	Traceability generated	Change impact traced
Risk classification	Safety reqs explicit	Design inputs locked	Tests by requirement	Audit log immutable	Audit trail preserved

Requirements Change. Trace the Change.

When a requirement evolves, the spec is updated first. The AI re-generates against the new spec. The delta is traceable. Vibe-coded systems have no equivalent — changes propagate invisibly.

Post-Market Surveillance Has a Home.

The audit log produced at runtime maps directly to your post-market surveillance obligations. Every state transition is timestamped, attributed, and immutable — by construction, not by retrofit.

THE ARGUMENT

In Regulated Healthcare, This Is the Only Way

Verification Is Not Optional

IEC 62304 requires demonstrable traceability from requirements through design through verification.

AI Makes Fragility Faster

Without constraints, AI generates plausible-but-wrong code at high velocity.

Specs Are Living Safety Artifacts

A requirement is not bureaucracy — it is a documented decision about patient safety.

The Industry Is Converging Here

AWS built spec-driven development into KIRO. GitHub ships SpecKit for Copilot. Independent practitioners arrive at the same conclusion — structure before syntax.

This is not a personal methodology. AWS · GitHub · Anthropic · Independent practitioners — all converging on the same model.

PRACTICAL PATH

Starting Monday: Three Scenarios.

01 NEW PROJECT

Run all four commands before writing a single line of code.

- specify init → claude → /speckit-constitution → /speckit-specify → /speckit-plan → /speckit-tasks
- Then: Implement by task ID (e.g. "Implement T020 from tasks.md")
- Result: spec, plan, tasks, implementation — all traceable from day one

02 EXISTING PROJECT

Pick the next feature. Run the four commands on that feature only.

- Don't retrofit 200,000 lines — start at the next feature boundary
- /speckit-constitution runs once per repo · /speckit-specify runs per feature
- Result: new work is traceable · old work stays as-is · gap closes over time

03 TEAM ADOPTION

Constitution goes in the repo. PR template enforces REQ-NNN.

- Commit constitution.md to .specify/memory/ — it's a version-controlled artifact
- Add REQ-NNN field to PR template · CI traceability check fails build if missing
- Result: every engineer follows spec-first automatically — no manual enforcement

Same four commands. Same prompts. Every time. Repeatable by design.

Three Truths.

01

AI doesn't make bad engineering good. It makes it faster.

02

A spec is not bureaucracy. It's the difference between building fast and building right fast.

03

Traceability isn't a compliance artifact. It's how you prove — to yourself, your team, and a regulator — that you knew what you were doing.

Lab651 — we responsibly leverage AI without compromising human oversight.